

SESSION 12

SESSION 23 — SEMAPHORES AND DEADLOCKS

EXPERIMENT 23

Aim

To study **semaphores and their operations** and demonstrate the use of a counting semaphore for **critical section management** in a multithreaded program.

Theory

A **semaphore** is a synchronization mechanism used to control access to shared resources by multiple processes or threads.

A semaphore maintains an integer value and provides two basic operations:

Wait / P Operation

The wait operation decreases the semaphore value. If the resource is unavailable, the thread waits.

```
wait(S)
```

Signal / V Operation

The signal operation increases the semaphore value and may wake a waiting thread.

signal(S)

Types of Semaphores

Binary Semaphore:

Can have values 0 or 1 and is commonly used for mutual exclusion.

Counting Semaphore:

Can have values greater than 1 and is used when multiple instances of a resource are available.

Critical Section

A critical section is the part of a program where a shared resource is accessed. Synchronization ensures that multiple threads do not incorrectly modify the shared resource at the same time.

Algorithm

1. Initialize a counting semaphore.
2. Create multiple threads.
3. Each thread requests access to the shared resource.
4. Perform the wait operation on the semaphore.
5. Enter the critical section.
6. Perform the required operation.
7. Leave the critical section.
8. Perform the signal operation.
9. Wait for all threads to complete.
10. Destroy the semaphore.

C Program:

```
C semaphore_demo.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <unistd.h>
6
7  #define NUM_THREADS 4
8
9  sem_t semaphore;
10
11 void *access_resource(void *arg)
12 {
13     int id = *(int *)arg;
14
15     printf("Thread %d is waiting for the resource.\n", id);
16
17     sem_wait(&semaphore);
18
19     printf("Thread %d entered the critical section.\n", id);
20
21     sleep(1);
22
23     printf("Thread %d leaving the critical section.\n", id);
24
25     sem_post(&semaphore);
26
27     return NULL;
28 }
29
30 int main()
31 {
32     pthread_t threads[NUM_THREADS];
33     int ids[NUM_THREADS];
34
35     // Initialize counting semaphore
36     // Two threads can access the resource at a time
37     sem_init(&semaphore, 0, 2);
38 }
```

```

30  int main()
31  {
32      pthread_t threads[NUM_THREADS];
33      int ids[NUM_THREADS];
34
35      // Initialize counting semaphore
36      // Two threads can access the resource at a time
37      sem_init(&semaphore, 0, 2);
38
39      printf("Counting semaphore initialized with value 2.\n\n");
40
41      for (int i = 0; i < NUM_THREADS; i++) {
42          ids[i] = i + 1;
43
44          pthread_create(&threads[i],
45                        NULL,
46                        access_resource,
47                        &ids[i]);
48      }
49
50      for (int i = 0; i < NUM_THREADS; i++) {
51          pthread_join(threads[i], NULL);
52      }
53
54      sem_destroy(&semaphore);
55
56      printf("\nAll threads completed successfully.\n");
57
58      return 0;
59  }

```

OUTPUT:

```
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc semaphore_demo.c -o semaphore_demo -pthread
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./semaphore_demo
Counting semaphore initialized with value 2.

Thread 2 is waiting for the resource.
Thread 3 is waiting for the resource.
Thread 3 entered the critical section.
Thread 4 is waiting for the resource.
Thread 1 is waiting for the resource.
Thread 2 entered the critical section.
Thread 3 leaving the critical section.
Thread 2 leaving the critical section.
Thread 4 entered the critical section.
Thread 1 entered the critical section.
Thread 4 leaving the critical section.
Thread 1 leaving the critical section.

All threads completed successfully.
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$
```

Result

Thus, semaphore synchronization was successfully implemented using a counting semaphore. The semaphore controlled access to the critical section and allowed a maximum of two threads to access the shared resource simultaneously.

SESSION 24 — PRODUCER-CONSUMER USING SEMAPHORES

EXPERIMENT 24

Aim

To implement the **Producer-Consumer problem using counting semaphores** and demonstrate synchronization between producer and consumer threads.

Theory

The **Producer-Consumer problem** is a classical synchronization problem.

The producer generates data and places it into a shared buffer, while the consumer removes data from the buffer.

Two counting semaphores are commonly used:

Empty

Keeps track of the number of empty spaces in the buffer.

Full

Keeps track of the number of occupied spaces.

A **mutex** is also used to ensure that only one thread accesses the buffer at a time.

Producer

↓

[Shared Buffer]

↓

Consumer

empty → available spaces

full → available items

mutex → protects buffer

Synchronization

If the buffer is full, the producer waits.

If the buffer is empty, the consumer waits.

This prevents race conditions and incorrect access to the shared buffer.

Algorithm

Producer

1. Wait for an empty buffer slot.
2. Lock the buffer using the mutex.
3. Insert an item.
4. Unlock the buffer.
5. Signal that an item is available.

Consumer

1. Wait for an available item.
2. Lock the buffer using the mutex.
3. Remove an item.
4. Unlock the buffer.
5. Signal that an empty slot is available.

C Program

```
C producer_consumer_sem.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <unistd.h>
6
7  #define BUFFER_SIZE 5
8  #define ITEMS 10
9
10 int buffer[BUFFER_SIZE];
11
12 int in = 0;
13 int out = 0;
14
15 sem_t empty;
16 sem_t full;
17
18 pthread_mutex_t mutex;
19
20 void *producer(void *arg)
21 {
22     for (int item = 1; item <= ITEMS; item++) {
23
24         sem_wait(&empty);
25
26         pthread_mutex_lock(&mutex);
27
28         buffer[in] = item;
29         in = (in + 1) % BUFFER_SIZE;
30
31         printf("Produced: %d\n", item);
32
33         pthread_mutex_unlock(&mutex);
34
35         sem_post(&full);
36
37         usleep(100000);
38     }
39
40     return NULL;
```

```

20 void *producer(void *arg)
40     return NULL;
41 }
42
43 void *consumer(void *arg)
44 {
45     for (int i = 1; i <= ITEMS; i++) {
46
47         sem_wait(&full);
48
49         pthread_mutex_lock(&mutex);
50
51         int item = buffer[out];
52         out = (out + 1) % BUFFER_SIZE;
53
54         printf("Consumed: %d\n", item);
55
56         pthread_mutex_unlock(&mutex);
57
58         sem_post(&empty);
59
60         usleep(150000);
61     }
62
63     return NULL;
64 }
65
66 int main()
67 {
68     pthread_t producer_thread;
69     pthread_t consumer_thread;
70
71     // Initialize semaphores
72     sem_init(&empty, 0, BUFFER_SIZE);
73     sem_init(&full, 0, 0);
74
75     pthread_mutex_init(&mutex, NULL);
76
77     printf("Starting Producer-Consumer using semaphores...\n\n");
78
79     pthread_create(&producer_thread,

```

```

71 // Initialize semaphores
72 sem_init(&empty, 0, BUFFER_SIZE);
73 sem_init(&full, 0, 0);
74
75 pthread_mutex_init(&mutex, NULL);
76
77 printf("Starting Producer-Consumer using semaphores...\n\n");
78
79 pthread_create(&producer_thread,
80              NULL,
81              producer,
82              NULL);
83
84 pthread_create(&consumer_thread,
85              NULL,
86              consumer,
87              NULL);
88
89 pthread_join(producer_thread, NULL);
90 pthread_join(consumer_thread, NULL);
91
92 sem_destroy(&empty);
93 sem_destroy(&full);
94 pthread_mutex_destroy(&mutex);
95
96 printf("\nProducer and Consumer completed successfully.\n");
97
98 return 0;
99 }

```

OUTPUT:

```
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc producer_consumer_sem.c -o
producer_consumer_sem -pthread
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./producer_consumer_sem
Starting Producer-Consumer using semaphores...

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Produced: 5
Consumed: 4
Produced: 6
Consumed: 5
Produced: 7
Produced: 8
Consumed: 6
Produced: 9
Consumed: 7
Produced: 10
Consumed: 8
Consumed: 9
Consumed: 10

Producer and Consumer completed successfully.
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$
```

Result

Thus, the Producer-Consumer problem was successfully implemented using counting semaphores and a mutex. The semaphores synchronized the producer and consumer threads, preventing buffer overflow, buffer underflow, and race conditions.